

SQL 是自含式(交互式)语言、嵌入式语言、结构化查询语言
集 DDL、DML、DCL 功能为一体

DDL 数据定义语言 { 创建定义 CREATE
删除定义 DROP
修改定义 ALTER

DML 数据操纵语言 { 数据查询 SELECT
数据插入 INSERT
数据更新 UPDATE
数据删除 DELETE

DCL 数据控制语言 { 授予权限 GRANT
收回权限 REVOKE

MySQL 默认的存储引擎是 InnoDB

InnoDB 的优势: 更新、删除、并发性、事务完整性

(事务型存储引擎, 支持行级锁定以及外键完整性约束)

MyISAM 的优势: 插入、查询

(与 InnoDB 相反)

查看当前 MySQL 支持的存储引擎: SHOW ENGINES;

查看默认存储引擎: SHOW VARIABLES LIKE '% default-storage-engine %';

修改默认存储引擎: SET DEFAULT-STORAGE-ENGINE = ... ;

当前MySQL服务器是否支持该类型存储引擎

是否支持分布式交易处理的XA规范

存储引擎的类型

解释

是否支持事务

是否支持事务处理中的保存点

Engine	Support	Comment	Transactions	XA	Savepoints
MEMORY	YES	Hash based, stored	NO	NO	NO
MRG_MYISAM	YES	Collection of iden	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
FEDERATED	NO	Federated MySQL st (NZZL)	(NZZL)	(NZZL)	(NZZL)
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
MyISAM	YES	MyISAM storage eng	NO	NO	NO
InnoDB	DEFAULT	Supports transacti	YES	YES	YES
BLACKHOLE	YES	/dev/null storage	NO	NO	NO
ARCHIVE	YES	Archive storage en	NO	NO	NO

当前MySQL服务器默认的存储引擎

查看当前存储引擎: SHOW VARIABLES LIKE '% storage-engine %';

字符集

服务器与客户端必须使用相同的编码和解码规则

↑ ↓
存储 显示
(编码) (解码)

例如 例如
gbk-chinese-ci 大小写不敏感
与 gbk-chinese-cs 大小写敏感
一个字符集可包含多种字符序, 每个字符集有一个默认字符序,
每个字符序只能对应唯一的字符集, 不能跨字符集混用

查看当前MySQL支持的字符集与默认字符序: SHOW CHARACTER SET; (SHOW CHARSET;)

查看当前MySQL使用的字符集: SHOW VARIABLES LIKE 'character%';

UTF8 中文按偏旁索引顺序排序, GBK 中文按字母顺序排序

中文简字
支持中文简体的字符集 {
GBK 与 GB2312 : 2 个字节
(GBK 是 GB2312 超集)
UTF-8 : 3 个字节 (utf8mb4 是 UTF-8 超集)
GB18030 : 4 个字节 (emoji 超集)

将 student 表的 sname 字段由 utf8 字符集转为 gbk 排序:

目前MySQL支持41种字符集

SELECT * FROM student ORDER BY CONVERT(sname USING gbk);

创建数据库

CREATE DATABASE IF NOT EXISTS db-name -- 如 student
设置默认字符集、字符序 {
不写则采用系统默认 CHARACTER SET charset-name -- 如 utf8mb4
COLLATE : collation-name ;
如 gbk-chinese-ci

查看数据库

所有: SHOW DATABASES;

某个: SHOW CREATE DATABASE db-name ; -- 如 student

使用数据库

切换为指定数据库: USE db-name ; -- 如 student

查看当前数据库: SELECT DATABASE();

修改数据库

ALTER : DATABASE db-name -- 如 student
DEFAULT CHARACTER SET charset-name -- 如 utf8
COLLATE collation-name ; -- 如 gbk-chinese-ci

删除数据库

DROP DATABASE IF EXISTS db-name ; -- 如 student

数字类型

- 表的主键建议使用整数类型, 处理效率高, 查询速度快
- 定义无符号数据类型时需要在数据类型右边加 **UNSIGNED** 关键字
- 人为指定的显示宽度与取值范围无关, 若数据位数小于显示宽度就用空格填充, 若大于显示宽度也不影响显示结果

当为字段设置零填充 **ZEROFILL** 时, 若数据位数小于显示宽度就会在左侧补0

① 创建表并设置字段零填充 ② 插入数据 ③ 查询数据

```
CREATE TABLE mytable  
(  
  int1 属性名 INT ZEROFILL,  
  int2 INT(6) ZEROFILL  
);
```

```
INSERT INTO mytable VALUES (123, 123);
```

```
SELECT * FROM mytable;
```

结果:

int1	int2
0000000123	000123

- 定义小数时尽可能使用定点数类型 **DECIMAL** (**FLOAT**, **DOUBLE PRECISION** 浮点数类型有精度损失)

字符串类型

每个字符占一字节

- CHAR** 类型的存储字长是固定的, 若实际数据不足会自动在后面补0填满
- VARCHAR** 类型的存储字长是可变的, 取决于实际数据的字节长度, 存储最大长度取决于字符集
- TEXT** 类型 (长文本) 的执行效率不如 **CHAR** 类型和 **VARCHAR** 类型 **不区分大小写**
- BLOB** 类型 (二进制长文本) 保存图片、PDF 文档等. **区分大小写**

日期和时间类型

- YEAR** 类型: YYYY
- DATE** 类型: YYYY-MM-DD **CURRENT-DATE** 返回系统当前日期 **NOW()** 返回系统当前日期、时间
- TIME** 类型: HH:MM:SS **CURRENT-TIME** 返回系统当前时间
- DATETIME** 类型: YYYY-MM-DD HH:MM:SS
- TIMESTAMP**: YYYY-MM-DD HH:MM:SS **CURRENT-TIMESTAMP** 返回系统当前日期、时间
该类型取值范围小于 **DATETIME**, 字段一般自动设置成非空 **NOT NULL**

ENUM 类型 (枚举类型) 类似单选框

SET 类型 类似复选框

查询速度比
CHAR、VARCHAR 更快

```
CREATE TABLE myenum  
(  
  myname VARCHAR(20),  
  gender ENUM('男', '女')  
);
```

```
INSERT INTO myenum VALUES ('张三', '男'), ('李四', '女');
```

```
SELECT * FROM myenum;
```

```
CREATE TABLE myset  
(  
  myname VARCHAR(20),  
  hobby SET('吃', '喝', '玩')  
);
```

```
INSERT INTO myset VALUES ('张三', '吃,喝');
```

```
SELECT * FROM myset WHERE hobby = '吃,喝';
```

JSON 类型

```
CREATE TABLE myjson ( j1 json, j2 json );  
INSERT INTO myjson VALUES ( ' { "k1": "value", "k2": 10 } ', ' [ "run", "sing", 10, null, true ] ' );  
SELECT * FROM myjson;
```

JSON 对象

JSON 数组

键 值 键值对
j1

{ "k1": "value", "k2": 10 } j2
["run", "sing", 10, null, true]

数据表管理

创建数据表

删除数据表: DROP TABLE *t-name*;

如 mytable

在 teaching-manage 数据库中创建一个名为 student 的数据表, 用于保存学生信息
course
score
成绩

```
CREATE TABLE IF NOT EXISTS student
```

```
(  
  Sno CHAR(6) PRIMARY KEY COMMENT '学生学号',  
  Sname VARCHAR(20) NOT NULL COMMENT '学生姓名',  
  dept VARCHAR(20) COMMENT '专业',  
  Sex CHAR(1) COMMENT '性别',  
  birthday DATE COMMENT '出生日期',  
  totalcredit DECIMAL(4,1) DEFAULT 0 COMMENT '总学分',  
  remarks VARCHAR(100) COMMENT '备注'  
)
```

总位数

// 4位, 小数点后1位

不输入值时用默认值0填充

```
ENGINE = INNODB
```

// 存储引擎

```
DEFAULT CHARSET = utf8mb4
```

// 字符集

```
COLLATE = utf8mb4-0900-ai-ci;
```

// 字符序, 大小写不敏感 (校对集)

```
CREATE TABLE IF NOT EXISTS course
```

```
(  
  cno CHAR(3) PRIMARY KEY COMMENT '课程号',  
  cname VARCHAR(30) NOT NULL UNIQUE COMMENT '课程名称',  
  term TINYINT COMMENT '开课学期',  
  ctime TINYINT UNSIGNED COMMENT '课时',  
  credit DECIMAL(3,1) COMMENT '学分'  
)
```

每行课程名不可重复

```
CREATE TABLE IF NOT EXISTS score
```

```
(  
  score_id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY '成绩表id',  
  Sno CHAR(6) COMMENT '学生学号, 外键',  
  Cno CHAR(3) COMMENT '课程号, 外键',  
  grade DECIMAL(4,1) CHECK(grade >= 0) COMMENT '成绩',  
  CONSTRAINT fk_score_sno FOREIGN KEY (sno) REFERENCES student (sno),  
  CONSTRAINT fk_score_cno FOREIGN KEY (cno) REFERENCES course (cno)  
)
```

针对外键的

参照完整性
约束

```
CREATE TABLE SC.
```

利用 score 创建数据表 SC 用于保存成绩信息

AS SELECT * FROM score; "CREATE TABLE + 子查询"方式不能复制约束和索引, 只能复制表结构和内容

查看数据表(当前数据库)

查看所有: SHOW TABLES;

查看以'stu'开头的: SHOW TABLES LIKE 'stu%';

匹配模式符: "%" 后跟0个或多个字符均可匹配, "_" 仅可匹配单个字符(a-可匹配ab.ac...)

查看数据表详细信息

查看当前数据库所有数据表信息: SHOW TABLE STATUS;

查看指定数据库所有数据表信息: SHOW TABLE STATUS FROM db-name;
如 student

查看数据表结构

查看当前数据库中某表结构: DESCRIBE t-name; 或 SHOW COLUMNS FROM t-name;
如 mytable 如 mytable

查看当前数据库中某表中某字段结构: DESCRIBE t-name z-name;
如 mytable 如 myzidian

查看数据表创建语句

查看当前数据库中某表创建语句: SHOW CREATE TABLE t-name;
如 mytable

修改数据表名

ALTER TABLE ^{如 mytable1} 原表名 RENAME TO ^{如 mytable2} 新表名 ;

修改数据表选项

ALTER TABLE ^{如 mytable} 表名 CHARSET = ^{如 utf8} charset-name ;

修改数据表结构

增加新列: ALTER TABLE ^{如 mytable} 表名 ADD COLUMN ^{如 mylie} 列名 ^{如 VARCHAR(50)} 类型 ;

在第一个位置增加新列: ALTER TABLE ^{如 mytable} 表名 ADD ^{如 mylie} 列名 ^{如 VARCHAR(50)} 类型 FIRST ;

在指定列后增加新列: ALTER TABLE ^{如 mytable} 表名 ADD ^{如 mylie2} 列名 ^{如 mylie1} 类型 AFTER 已有列 ;

修改列:

→ 改列名: ALTER TABLE ^{如 mytable} 表名 CHANGE ^{如 mylie1} 原列名 ^{如 mylie2} 新列名 ^{如 VARCHAR} 类型 ;

→ 改类型和位置: ALTER TABLE ^{如 mytable} 表名 MODIFY ^{如 mylie2} 列名 ^{如 VARCHAR(20)} 新类型 AFTER ^{如 mylie1} 已有列 ;

→ 删除列: ALTER TABLE ^{如 mytable} 表名 DROP ^{如 mylie} 列名 ;

添加和删除约束:

给 score 的 sno、cno 两列添加唯一性约束

ALTER TABLE score ADD CONSTRAINT uk_sno_cno UNIQUE (sno, cno)

给 score 的 sno、cno 两列删除唯一性约束

ALTER TABLE score DROP CONSTRAINT uk_sno_cno ;

↓
有约束名

无约束名:

例如前面 course 表中, 只定义了唯一性约束却没有命名

Cname VARCHAR(30) NOT NULL UNIQUE COMMENT '课程名'

(要想有约束名, 应加一句 CONSTRAINT uniq_cname UNIQUE (cname),)

就以列名作约束名: 把上面的 UNIQUE 去掉,

ALTER TABLE course DROP CONSTRAINT cname;

插入数据 (字符串类型、日期和时间类型的数据插入时加 '')

插入一条记录

所有字段都有值: `INSERT INTO student VALUES ('001242', '张青', '计算机', '女', '04-1-22', 50, '三好学生');`

部分字段有值: `INSERT INTO student (sno, sname, dept) VALUES ('001243', '李进', '化工');`
未出现的列值为 NULL, 表定义时说明了 NOT NULL 的属性列必须有值! --对应

插入多条记录

`INSERT INTO score VALUES (NULL, '001243', '101', 90), (NULL, '001243', '102', 90);`

插入子查询返回的结果集

```
CREATE TABLE stu
(
  sno CHAR(6),
  sname VARCHAR(10),
  dept VARCHAR(20),
  sex VARCHAR(4)
);

INSERT INTO stu
SELECT sno, sname, dept, sex
FROM student
WHERE dept = '计算机';
```

修改数据

无 WHERE, 更新表中所有行: `UPDATE student SET totalcredit = totalcredit + 100;`
有 WHERE, 更新条件表达式值为 True 的行:

`UPDATE student SET dept = '计算机', remarks = '三好学生' WHERE sname = '罗茜茜';`

删除数据

无 WHERE, 删除所有记录, 但表定义仍存在: `DELETE FROM student;` 不能释放存储空间

有 WHERE, 删除满足条件表达式的记录: `DELETE FROM student WHERE dept = '计算机';`

TRUNCATE TABLE t_name 语句截断基本表, 删除表中所有行并释放空间, 执行后不可撤销 更快

若表是一个引用完整性的父表,

那么想要完成截断操作就必须在 TRUNCATE TABLE 语句前禁用约束.

AUTO-INCREMENT 是MySQL唯一扩展的完整性约束

两个参数：
{ 增量初始值(默认为1) auto-increment-offset (1~65535)
{ 增量幅度(默认为1) auto-increment-increment (1~65535)

设置与查看: SET @@ auto-increment-offset = 2;
SET @@ auto-increment-increment = 2;

SHOW VARIABLES LIKE 'auto-inc%';

一个数据表中只能有一个字段使用该约束,且该字段值必为无符号整数

例如, Score_id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY

插入数据

每插入一条记录,该字段值自动加1

```
CREATE TABLE stu  
(  
  stu-id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY,  
  sname VARCHAR(10)  
);  
INSERT INTO stu VALUES (NULL, '张三'), (NULL, '李四'), (10, '王五');
```

stu-id	sname
2	张三
4	李四
10	王五

当 AUTO-INCREMENT 类型的字段在插入时不为空,若所给值在表中已存在则插入记录无效,否则按当前数据插入,后续在当前值最大值上自增.

修改数据

通过修改表结构重新定义初始值时,原表中的数据不变化,只对新插入数据起效

```
CREATE TABLE stu  
(  
  stu-id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY,  
  sname VARCHAR(10)  
) AUTO-INCREMENT = 200001;  
INSERT INTO stu VALUES (NULL, '张三'), (NULL, '李四');  
ALTER TABLE stu AUTO-INCREMENT = 202201;  
INSERT INTO stu VALUES (NULL, '张三'), (NULL, '李四');
```

stu-id	sname
200001	张三
200002	李四

stu-id	sname
200001	张三
200002	李四
202201	张三
202202	李四

删除数据

使用DELETE,再插入新记录时仍从原序列最大值基础上自动增加

```
DELETE FROM stu;  
INSERT INTO stu VALUES (NULL, '张三'), (NULL, '李四');
```

stu-id	sname
202203	张三
202204	李四

使用 TRUNCATE ^{删除表中所有行(记录)},从初始值开始自增

```
TRUNCATE TABLE stu;  
INSERT INTO stu VALUES (NULL, '张三'), (NULL, '李四');
```

stu-id	sname
1	张三
2	李四

释放内存, stu-id 变量保存值.定义表或修改表设置的初始值都被清除.

重新插入记录时,系统使用全局变量设置的初始值填充自增型字段

auto-increment-offset 与 auto-increment-increment

完整性约束名	定义关键字	约束规则
实体完整性约束	PRIMARY KEY	主键不能取重复值和空值
参照完整性约束	FOREIGN KEY	外键的值依赖主键的值或是空值
检查约束	CHECK	满足表达式的值
非空约束	NOT NULL	不能取空值
唯一性约束	UNIQUE	不能有重复值
默认值约束	DEFAULT	不指定值时取默认值

要保证一个表的每行取值唯一, 实现实体完整性, 必须指定一列或多列作表的主键

主键值要唯一且非空!

列级定义

```
CREATE TABLE XS
( 学号 CHAR(9) PRIMARY KEY
  姓名 CHAR(8) NOT NULL,
  ...);
```

表级定义

```
CREATE TABLE XS
( 学号 CHAR(9),
  姓名 CHAR(8) NOT NULL,
  PRIMARY KEY(学号));
```

保证参照完整性的3个方法:

RESTRICT (缺省选择项, 限制引用操作) 不能对被子表引用的主键进行任何操作

CASCADE (级联引用操作) 子表中引用主键的行随主表主键而变化

SET NULL (置空操作) 当对主表的主键进行操作导致主键与外键值不一致时, 子表中引用该主键的值置 NULL

```
CREATE TABLE dept
( dno CHAR(3) PRIMARY KEY,
  dname VARCHAR(50));
```

```
CREATE TABLE emp
( eno CHAR(3) PRIMARY KEY,
  ename VARCHAR(50),
  dno CHAR(3),
  CONSTRAINT fk-dno FOREIGN KEY(dno) REFERENCES dept(dno);
```

用户自定义完整性:

```
CREATE TABLE XS
( sno CHAR(6) PRIMARY KEY,
  sname VARCHAR(10) UNIQUE,
  dept VARCHAR(20) NOT NULL,
  sex VARCHAR(4) DEFAULT('男'),
  totalcredit DECIMAL(3,0) CHECK(TOTALCREDIT >= 0);
```

3位的整数(定点)

约束命名

```
CREATE TABLE XS
( sno CHAR(6) PRIMARY KEY,
  sname VARCHAR(10),
  dept VARCHAR(20) NOT NULL,
  sex VARCHAR(4) DEFAULT('男'),
  totalcredit DECIMAL(3,0),
  CONSTRAINT uk_sname UNIQUE(sname),
  CONSTRAINT ck_credit CHECK(TOTALCREDIT >= 0);
```

删除原约束条件: ALTER TABLE XS DROP CONSTRAINT uk_sname;

添加新约束条件: ALTER TABLE XS ADD CONSTRAINT uk_sname UNIQUE(sname);